

1. Singleton Pattern:

Implementation Theory

The Logger class is implemented using the Singleton design pattern to ensure that only one instance of the logger exists throughout the application. The constructor is made private to prevent object creation from outside the class, and a static method returns the single instance whenever required. This provides centralized and consistent logging across the application.

Execution

```
using System;

namespace DesignPatternsExercise
{
    public sealed class LoggerSingleton
    {

        private static readonly Lazy<LoggerSingleton> _instance =
            new Lazy<LoggerSingleton>(() => new LoggerSingleton());

        public static LoggerSingleton Instance => _instance.Value;

        private LoggerSingleton()
        {
            Console.WriteLine("LoggerSingleton Initialized.");
        }

        public void LogMessage(string message)
        {
            Console.WriteLine($"[{DateTime.Now:yyyy-MM-dd HH:mm:ss}] LOG: {message}");
        }
    }
}
```

```
    }  
}  
  
class SingletonProgram  
{  
    static void Main(string[] args)  
    {  
        Console.WriteLine("--- Testing Singleton Pattern ---");  
  
        LoggerSingleton logger1 = LoggerSingleton.Instance;  
        logger1.LogMessage("Application started.");  
  
        LoggerSingleton logger2 = LoggerSingleton.Instance;  
        logger2.LogMessage("Performing database operation.");  
  
        if (ReferenceEquals(logger1, logger2))  
        {  
            Console.WriteLine("\nSUCCESS: Both logger variables point to the exact same  
instance.");  
        }  
        else  
        {  
            Console.WriteLine("\nFAILURE: Instances are different.");  
        }  
    }  
}
```

```
--- Testing Singleton Pattern ---
LoggerSingleton Initialized.
[2026-06-21 06:08:34] LOG: Application started.
[2026-06-21 06:08:34] LOG: Performing database operation.

SUCCESS: Both logger variables point to the exact same instance.

...Program finished with exit code 0
Press ENTER to exit console.[]
```

Factory Method Pattern:

Implementation Theory

The Factory Method Pattern is used to create different types of documents without directly instantiating their classes. An abstract DocumentFactory defines the createDocument() method, and concrete factory classes implement this method to create specific document types such as Word, PDF, and Excel documents. This approach promotes loose coupling, flexibility, and easy extension of the system by adding new document types without modifying existing code.

Execution

using System;

namespace DesignPatternsExercise

```
{
    public interface INotification
    {
        void Send(string message);
    }
}
```

```
public class EmailNotification : INotification
{
    public void Send(string message)
    {
        Console.WriteLine($"Email sent with content: '{message}'");
    }
}
```

```
public class SmsNotification : INotification
{
    public void Send(string message)
    {
        Console.WriteLine($"SMS sent with content: '{message}'");
    }
}
```

```
public abstract class NotificationFactory
{
    public abstract INotification CreateNotification();

    public void DeliverNotification(string message)
    {
        INotification notification = CreateNotification();
        notification.Send(message);
    }
}
```

```
}
```

```
public class EmailNotificationFactory : NotificationFactory
```

```
{
```

```
    public override INotification CreateNotification()
```

```
    {
```

```
        return new EmailNotification();
```

```
    }
```

```
}
```

```
public class SmsNotificationFactory : NotificationFactory
```

```
{
```

```
    public override INotification CreateNotification()
```

```
    {
```

```
        return new SmsNotification();
```

```
    }
```

```
}
```

```
class FactoryMethodProgram
```

```
{
```

```
    static void Main(string[] args)
```

```
    {
```

```
        Console.WriteLine("--- Testing Factory Method Pattern ---");
```

```
        NotificationFactory emailFactory = new EmailNotificationFactory();
```

```
        emailFactory.DeliverNotification("Welcome to Cognizant C# .NET Training!");
```

```
        Console.WriteLine();
```

```
NotificationFactory smsFactory = new SmsNotificationFactory();  
smsFactory.DeliverNotification("Your OTP is 482109.");  
}  
}  
}
```

O/p:

```
--- Testing Factory Method Pattern ---  
Email sent with content: 'Welcome to Cognizant C# .NET Training!'  
  
SMS sent with content: 'Your OTP is 482109.'  
  
...Program finished with exit code 0  
Press ENTER to exit console.□
```